

Problem, Team, and Overview of Experiments

Build a distributed microservices architecture that isolates failures, scales automatically, and maintains performance during traffic spikes.

- User Service: Zhixiao Wu
- Social-Graph Service: Yi Xu
- Post Service: Bijing Tang
- Timeline Service: Wenshuang Zhou

Experiments Metrics:

- Post creation latency
- Timeline generation time
- Database operations count
- Storage overhead
- Throughput scaling

Project Plan and Recent Progress

Project Plan

Phase 1: Parallel Development (Week 1-2)

Each person develops their individual service Define inter-service API contracts

Phase 2: Integration & Testing (Week 3)

Person 4 coordinates inter-service calls Service mesh setup and testing

Phase 3: Experiment Execution (Week 4)

Recent Progress

- User Service: Finished service development using rds; Implemented gRPC; Tested API connection and successfully deployed on AWS.
- Social-Graph Service: Defines tier-based segmentation (small, medium, big, top), generates following/follower relationships using power law rules; Utilizes DynamoDB for storing user relationship data.; API endpoints are functional: followers/following lists, relationship checks, etc; Integrated ECS, ECR, ALB, and DynamoDB via Terraform modules, deployed successfully on AWS Innovation Sandbox.
- Post Service:
- Timeline Service: Finished main logic development; tested API connected; Deployed successfully on AWS Innovation Sandbox.

Objectives

Short-term Objective

Deploy the whole system successfully on AWS. Verifying all APIs are working correctly. Finishing a whole process from creating a user, following some other users, posting a new Post, fetching the latest posts in timeline.

Long-term Objective

Finishing the 3 algorithms experiments. Verifying the system can handle spike traffics.

Related work

Distributed Systems Fundamentals:

- CAP Theorem and its implications
- Consistency models in distributed databases
- Microservices architecture patterns
- Load balancing

Social Network Strategies

- Push vs Pull vs Hybrid approaches

Real-world Implementations:

- Service Discovery for RPC.

Methodology

Experiment 1: Push Model Fan-out

Post Creation Process

User creates a new post through REST API System writes post reference to each follower's personal timeline table

Timeline Retrieval Process

User requests timeline through GET /timeline/{user_id} System queries user's pre-computed timeline table
System returns posts in chronological order

Data Collection Points

Post creation latency (end-to-end time) Database write operations count per post Timeline read latency per user type Storage overhead per user (timeline table size)

Experiment 2: Pull Model Fan-out

Post Creation Process

User creates a new post through REST API System stores post only in central posts table with author_id
System returns success immediately after single database write

Timeline Retrieval Process

User requests timeline through GET /timeline/{user_id} System queries following table to get list of followed users System queries posts table for recent posts from each followed user System aggregates and sorts posts by timestamp System returns merged timeline

Data Collection Points

Post creation latency (minimal - single write) Timeline generation time per user type Database read operations count per timeline request Storage overhead (post table size)

Experiment 3: Hybrid Model Fan-out

Post Creation Process (Multi-path) For Regular Users and Influencers (< 50K followers):

Follow push-based process: write to all follower timelines

For Celebrities (> 50,000 followers):

Follow pull-based process: write only to posts table No timeline pre-computation

Timeline Retrieval Process (Multi-path)

For users following mostly Regular users:

Read both pre-computed timeline database and real-time aggregation from posts service

Merge and sort results by timestamp

Data Collection Points

Post creation latency Timeline generation time per user type Database read operations count per timeline request Storage overhead (post table size)

Preliminary Results

Impact

Startup and Scale-up Guidance. Early-stage social media companies face critical architectural decisions with limited resources for experimentation. Our results provide clear, data-driven recommendations for when to choose each fan-out strategy based on user base size, growth rate, and demographic distribution. This could prevent costly architectural mistakes that have historically forced complete system rewrites.

Cloud Cost Optimization. With concrete data on storage overhead, compute requirements, and scaling characteristics, companies can make informed decisions about infrastructure costs. Our analysis of cost-per-user across different algorithms and scales directly impacts bottom-line business decisions for social platforms.

Enterprise Social Platform Design

B2B social collaboration tools, internal communication platforms, and enterprise social networks can apply our findings to their specific use cases. The principles extend beyond consumer social media to any system requiring distributed timeline generation or activity feeds.